1. (2 Punkte) Es sei $f(n)$ die Laufzeit eines Algorithmus in Abhängigkeit von der Eingabe n. Bestimme die zugehörige Komplexitätsklasse.

   a) $f(n) = 2n \cdot \log n$
   b) $f(n) = 3n^2 + 4 \cdot 3^n$

   Lösung: a) $f(n) \in O(n \cdot \log(n))$ b) $f(n) \in O(3^n)$

2. (2 Punkte) Es sei $f(n)$ die Laufzeit eines Algorithmus in Abhängigkeit von der Eingabe n. Bestimme die zugehörige Komplexitätsklasse.

   a) $f(n) = 3n \cdot \log n + n$
   b) $f(n) = 5 + 6 + ... + n$

   Lösung: a) $f(n) \in O(n \cdot \log(n))$ b) $f(n) \in O(n^2)$

3. (4 Punkte) Es sei $f(n)$ die Laufzeit eines Algorithmus in Abhängigkeit von der Eingabe n. Bestimme die zugehörige Komplexitätsklasse.

   a) $f(n) = n^2 + 2n \cdot \log n$
   b) $f(n) = 3n^2 \cdot \log n + 0.5 \cdot 2^n$
   c) $f(n) = \log(n^2)/3$
   d) $f(n) = \log(n) + n$

   Lösung: a) $f(n) \in O(n^2)$ b) $f(n) \in O(2^n)$ c) $f(n) \in O(\log n)$ d) $f(n) \in O(n)$

4. (3 Punkte) Ein Programm braucht für eine Eingabe der Größe $n = 1000$ eine Laufzeit von 2 Sekunden.
   (a) Schätze die Laufzeit für $n = 2000$ und $n = 10\,000$, wenn die Laufzeit in $O(n)$, $O(n^2)$ bzw. $O(n^3)$ liegt.
   (b) Ein anderes Programm mit einer Laufzeit in $O(2^n)$ braucht für $n = 30$ eine Sekunde. Wie lange braucht es ungefähr für $n = 40$?

   Lösung: a)

	$n = 2000$	$n = 10\,000$
$O(n)$	4 s	20 s
$O(n^2)$	8 s	200 s $\approx$ 3 min
$O(n^3)$	16 s	2000 s $\approx$ 33 min

   b) $2^{10} \approx 1000$, also etwa 1000 s $\approx$ 17 min.

5. (2 Punkte) In welcher Komplexitätsklasse ist die Laufzeit der Funktion `foo` in Abhängigkeit von der Länge n der übergebenen Liste a im best case und im worst case?

```
def foo(a):
    n = len(a)
    for i in range(n):
        if a[i] == 7:
            for j in range(n):
                for k in range(n):
                    print("hello")
```

   Lösung: best case: $O(n)$, worst case: $O(n^3)$

6. (2 Punkte) In welcher Komplexitätsklasse ist die Laufzeit der Funktion `foo` in Abhängigkeit von der Länge n der übergebenen Liste a im best case und im worst case?

```
def foo(a):
    n = len(a)
    for i in range(n-1):
        for j in range(i+1,n):
            if a[i] == a[j]:
                return True
    return False
```

Lösung: best case: $O(1)$, worst case: $O(n^2)$